

Chapter 05 · SQL Databases

Sub-Chapter 08 — Table Partitioning

08 / 10

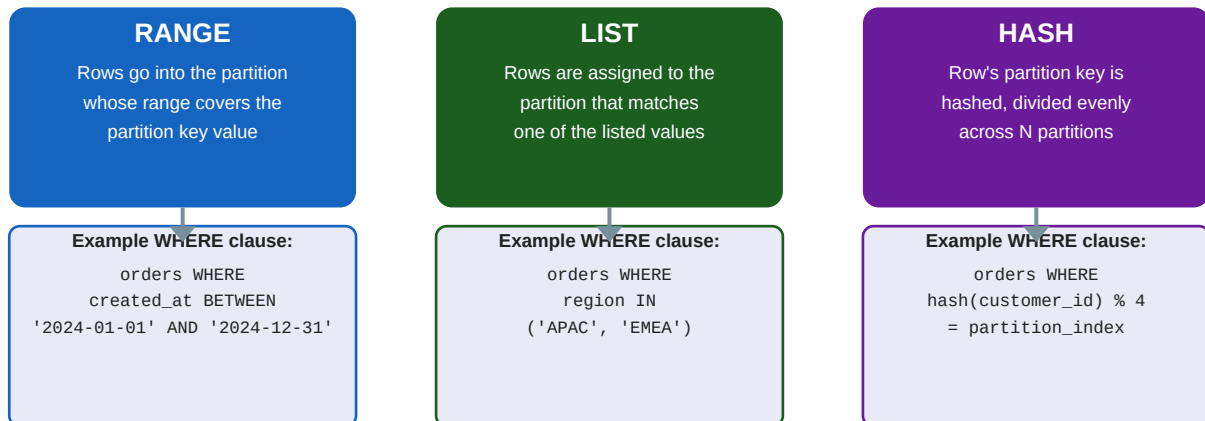
Scope	RANGE, LIST, HASH partitioning; pruning; maintenance
Database	PostgreSQL declarative partitioning
Level	Intermediate → Advanced
Outcome	Design, manage, and automate partitioned tables in Spring

1 What Is Table Partitioning?

Splitting one logical table into smaller physical pieces

A **partitioned table** looks like a single table to application code but is physically split into multiple child tables called **partitions**. The database routes each row to the correct partition automatically based on a **partition key** (e.g., a date column or region code). The two main reasons to partition are:

PostgreSQL Declarative Partitioning — Three Strategies



Query performance (pruning) — The planner skips partitions whose range cannot contain matching rows — a query on 2024 data never touches 2022 files.

Bulk data management — Drop an entire year of data by dropping one child partition — O(1) DDL vs O(millions) DELETE.

Index size — Each partition has its own indexes, which stay small and hot in cache.

Parallel query — PostgreSQL can scan multiple partitions in parallel (`enable_parallel_append = on`).

2 RANGE Partitioning

Most common strategy — partition by date or numeric range

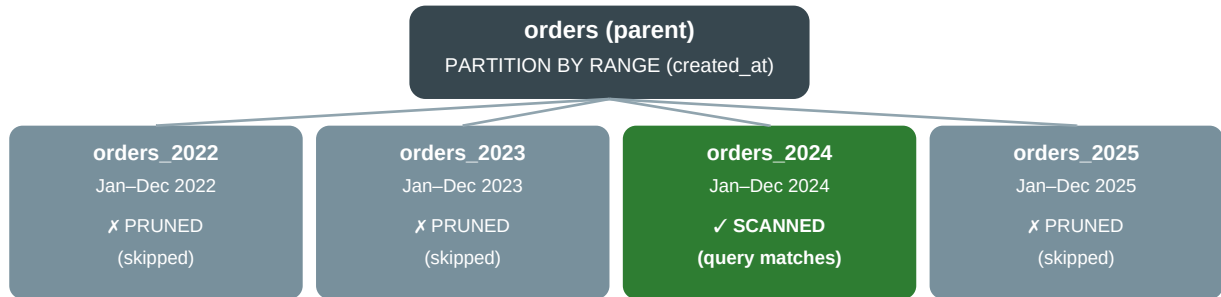
Range partitioning is ideal for time-series data (orders, events, logs). Each partition covers a non-overlapping interval.

2.1 Creating a Range-Partitioned Table

```
-- Parent table – no data stored here CREATE TABLE orders ( id BIGSERIAL, customer_id INT NOT NULL,
total NUMERIC NOT NULL, status TEXT NOT NULL DEFAULT 'PENDING', created_at TIMESTAMPTZ NOT NULL
DEFAULT NOW() ) PARTITION BY RANGE (created_at); -- Child partitions – one per year CREATE TABLE
orders_2023 PARTITION OF orders FOR VALUES FROM ('2023-01-01') TO ('2024-01-01'); CREATE TABLE
orders_2024 PARTITION OF orders FOR VALUES FROM ('2024-01-01') TO ('2025-01-01'); CREATE TABLE
orders_2025 PARTITION OF orders FOR VALUES FROM ('2025-01-01') TO ('2026-01-01'); -- Indexes on
each partition (NOT on the parent) CREATE INDEX idx_orders_2024_customer ON orders_2024
(customer_id); CREATE INDEX idx_orders_2025_customer ON orders_2025 (customer_id); -- Verify
partition structure SELECT parent.relname AS parent, child.relname AS partition,
pg_get_expr(child.relpartbound, child.oid) AS bounds FROM pg_inherits JOIN pg_class parent ON
pg_inherits.inhparent = parent.oid JOIN pg_class child ON pg_inherits.inhrelid = child.oid WHERE
parent.relname = 'orders';
```

2.2 Partition Pruning in Action

Partition Pruning — Only Scan Relevant Partitions



```
SELECT * FROM orders WHERE created_at BETWEEN '2024-01-01' AND '2024-12-31';
```

→ Planner sees constraint: only orders_2024 can contain rows in that range

→ orders_2022, orders_2023, orders_2025 are pruned — zero I/O on those files

```
-- Planner prunes 3 out of 4 partitions: EXPLAIN (ANALYZE, BUFFERS) SELECT COUNT(*) FROM orders
WHERE created_at BETWEEN '2024-06-01' AND '2024-06-30'; -- EXPLAIN output shows: -- Append -- ->
Seq Scan on orders_2024 (cost=... rows=...) -- (Partitions pruned: 3)
```

2.3 Default Partition — Catch Unexpected Values

```
-- A DEFAULT partition absorbs rows that don't fit any other partition -- Without it, inserting a
row outside all ranges raises an error CREATE TABLE orders_default PARTITION OF orders DEFAULT; --
Check what landed in default (indicates missing partition): SELECT COUNT(*), MIN(created_at),
MAX(created_at) FROM orders_default;
```

3

LIST & HASH Partitioning

Partition by discrete values or distribute load evenly

3.1 LIST Partitioning — Partition by Region

```
CREATE TABLE orders_global ( id BIGSERIAL, customer_id INT NOT NULL, total NUMERIC NOT NULL,
region TEXT NOT NULL, created_at TIMESTAMPTZ NOT NULL DEFAULT NOW() ) PARTITION BY LIST (region);
CREATE TABLE orders_apac PARTITION OF orders_global FOR VALUES IN ('APAC', 'ANZ', 'INDIA'); CREATE
TABLE orders_emea PARTITION OF orders_global FOR VALUES IN ('UK', 'DE', 'FR', 'ES'); CREATE TABLE
orders_amer PARTITION OF orders_global FOR VALUES IN ('US', 'CA', 'BR', 'MX'); CREATE TABLE
orders_global_default PARTITION OF orders_global DEFAULT; -- Queries filtered by region only scan
the relevant partition: SELECT * FROM orders_global WHERE region = 'APAC' AND total > 1000;
```

3.2 HASH Partitioning — Even Distribution

When there's no natural range or list, HASH partitioning distributes rows uniformly across N partitions using a hash of the partition key. Useful for very large tables where you want parallel I/O.

```
CREATE TABLE sessions ( id UUID DEFAULT gen_random_uuid(), user_id BIGINT NOT NULL, data JSONB,
expires_at TIMESTAMPTZ ) PARTITION BY HASH (user_id); -- 4 equally-sized partitions CREATE TABLE
sessions_p0 PARTITION OF sessions FOR VALUES WITH (MODULUS 4, REMAINDER 0); CREATE TABLE
sessions_p1 PARTITION OF sessions FOR VALUES WITH (MODULUS 4, REMAINDER 1); CREATE TABLE
sessions_p2 PARTITION OF sessions FOR VALUES WITH (MODULUS 4, REMAINDER 2); CREATE TABLE
sessions_p3 PARTITION OF sessions FOR VALUES WITH (MODULUS 4, REMAINDER 3); -- Any query with
user_id in WHERE goes to exactly 1 partition: SELECT * FROM sessions WHERE user_id = 42;
```

4

Partition Maintenance

Adding, archiving, and dropping partitions efficiently

Partition Lifecycle — Add, Archive, Drop

CREATE PARTITION

Add next month's partition in advance (pg_cron or Spring Scheduler)

DETACH PARTITION

Remove partition from table without deleting data (instant, no lock on parent)

ATTACH PARTITION

Re-attach an archived partition back (e.g., for historical reporting query)

DROP PARTITION

Physically delete old data — single fast DROP TABLE on the child table

DROP TABLE on a child partition is O(1) — far faster than DELETE with millions of rows

4.1 Adding a New Partition

```
-- Proactively create next month's partition (run via pg_cron or Spring Scheduler) CREATE TABLE
orders_2026 PARTITION OF orders FOR VALUES FROM ('2026-01-01') TO ('2027-01-01'); -- Create index
on new partition immediately CREATE INDEX CONCURRENTLY idx_orders_2026_customer ON orders_2026
(customer_id);
```

4.2 Archiving — DETACH and Store Elsewhere

```
-- DETACH removes the partition from the parent table -- The child table remains with its data -- it
just becomes a standalone table ALTER TABLE orders DETACH PARTITION orders_2022 CONCURRENTLY; --
Now archive it: copy to cold storage table, S3, or just leave detached -- Option: move to a
tablespace on cheaper disk ALTER TABLE orders_2022 SET TABLESPACE cold_storage; -- Re-attach for
ad-hoc historical reporting: ALTER TABLE orders ATTACH PARTITION orders_2022 FOR VALUES FROM
('2022-01-01') TO ('2023-01-01');
```

4.3 Dropping Old Data — O(1) Operation

```
-- The old way (DO NOT do this for large tables): DELETE FROM orders WHERE created_at <
'2020-01-01'; -- slow, bloats WAL -- The partition way (instant): ALTER TABLE orders DETACH
PARTITION orders_2019 CONCURRENTLY; DROP TABLE orders_2019; -- single metadata op, frees disk
instantly -- Compare: deleting 100M rows takes minutes; DROP TABLE takes milliseconds
```

5 Sub-Partitioning & Composite Keys

Partition by multiple dimensions for very large tables

PostgreSQL supports **sub-partitioning** — partitioning a partition further. For example, partition by year at the top level, then by region within each year.

```
-- Top-level: RANGE by year CREATE TABLE events ( id BIGSERIAL, user_id INT, region TEXT,
event_type TEXT, ts TIMESTAMPTZ NOT NULL ) PARTITION BY RANGE (ts); CREATE TABLE events_2024
PARTITION OF events FOR VALUES FROM ('2024-01-01') TO ('2025-01-01') PARTITION BY LIST (region);
-- sub-partition by region -- Sub-partitions of events_2024 CREATE TABLE events_2024_apac
PARTITION OF events_2024 FOR VALUES IN ('APAC', 'ANZ'); CREATE TABLE events_2024_emea PARTITION OF
events_2024 FOR VALUES IN ('UK', 'DE', 'FR'); CREATE TABLE events_2024_amer PARTITION OF
events_2024 FOR VALUES IN ('US', 'CA', 'BR'); -- A query scoped to year + region touches exactly
ONE leaf partition: SELECT COUNT(*) FROM events WHERE ts BETWEEN '2024-06-01' AND '2024-06-30' AND
region = 'APAC';
```

Caution: Sub-partitioning increases DDL complexity. Use it only when the table is in the hundreds-of-GB range and query patterns consistently filter on both dimensions.

6 Spring & JPA with Partitioned Tables

Transparent usage — partitioning is invisible to JPA

From JPA's perspective, a partitioned table is just a table. The entity maps to the parent table name, and all queries route correctly. The key areas to watch are **index strategy**, **bulk inserts**, and **partition creation automation**.

6.1 Entity Mapping (No Change Needed)

```
@Entity @Table(name = "orders") // maps to the parent table public class Order { @Id
@GeneratedValue(strategy = GenerationType.IDENTITY) private Long id; @Column(name = "customer_id",
nullable = false) private Long customerId; @Column(nullable = false) private BigDecimal total;
@Column(name = "created_at", nullable = false) private Instant createdAt; @Column(nullable =
false) private String status; @PrePersist void prePersist() { if (createdAt == null) createdAt =
Instant.now(); } }
```

6.2 Efficient Bulk Insert with JDBC Batch

JPA's `saveAll()` triggers individual INSERT statements. For loading large batches into partitioned tables, use JDBC batch directly:

```
@Service @RequiredArgsConstructor public class OrderBulkService { private final JdbcTemplate jdbc;
public void bulkInsert(List<Order> orders) { String sql = "INSERT INTO orders (customer_id, total,
status, created_at) VALUES (?, ?, ?, ?)"; jdbc.batchUpdate(sql, new BatchPreparedStatementSetter()
{ @Override public void setValues(PreparedStatement ps, int i) throws SQLException { Order o =
orders.get(i); ps.setLong(1, o.getCustomerId()); ps.setBigDecimal(2, o.getTotal());
ps.setString(3, o.getStatus()); ps.setTimestamp(4, Timestamp.from(o.getCreatedAt())); } @Override
public int getBatchSize() { return orders.size(); } }); }
```

6.3 Auto-Creating Partitions with Spring Scheduler

```
@Component @RequiredArgsConstructor public class PartitionManager { private final JdbcTemplate
jdbc; // Runs on the 1st of every month at midnight @Scheduled(cron = "0 0 1 * * *") public void
createNextMonthPartition() { // Create 2 months ahead so there's always a valid partition
YearMonth next = YearMonth.now().plusMonths(2); String tableName = "orders_" + next.getYear() +
"_" + String.format("%02d", next.getMonthValue()); String fromDate = next.atDay(1).toString();
String toDate = next.plusMonths(1).atDay(1).toString(); String sql = String.format( "CREATE TABLE
IF NOT EXISTS %s PARTITION OF orders FOR VALUES FROM ('%s') TO ('%s')", tableName, fromDate,
toDate); jdbc.execute(sql); log.info("Created partition: {} ({} to {})", tableName, fromDate,
toDate); // Create index on new partition jdbc.execute(String.format( "CREATE INDEX IF NOT EXISTS
idx_%s_customer ON %s (customer_id)", tableName, tableName)); } }
```

7 Pitfalls & Best Practices

Common mistakes and how to avoid them

Missing default partition — Without a DEFAULT partition, inserting a row outside all defined ranges raises an error. Always create a DEFAULT partition.

Primary key must include partition key — In PostgreSQL, a PRIMARY KEY or UNIQUE constraint must include the partition key column. Otherwise you'll get: 'unique constraint on partitioned tables must include all partitioning columns'.

Indexes on parent vs child — Creating an index on the parent table automatically creates matching indexes on all children. But you can also create partition-specific indexes for column combinations only relevant to some partitions.

Foreign keys to partitioned tables — As of PostgreSQL 16, foreign keys referencing partitioned tables are supported but can be slow on large tables. Consider enforcing FK at application level.

ATTACH PARTITION validates existing data — When you ATTACH a standalone table as a partition, PostgreSQL scans all rows to verify they fit the partition bounds. Add a CHECK constraint first to skip the scan: ALTER TABLE t ADD CONSTRAINT c CHECK (ts >= '2022-01-01' AND ts < '2023-01-01') NOT VALID; then validate.

Partition key immutability — Updating the partition key of a row moves it to a different partition (PostgreSQL does this automatically via DELETE + INSERT). This is expensive for large updates — design partition keys to be immutable.

8 Quick-Reference Cheatsheet

One-liner SQL for every partitioning operation

Operation	SQL	Notes
Create range-partitioned table	CREATE TABLE t (...) PARTITION BY RANGE (col)	No data stored in parent
Add range partition	CREATE TABLE t_2025 PARTITION OF t FOR VALUES FROM ('2025-01-01') TO ('2026-01-01')	
Add list partition	CREATE TABLE t_apac PARTITION OF t FOR VALUES IN ('APAC', 'ANZ')	
Add hash partition	CREATE TABLE t_p0 PARTITION OF t FOR VALUES WITH (MODULUS 4, REMAINDER 0)	
Add default partition	CREATE TABLE t_default PARTITION OF t DEFAULT	Catches unmatched rows
Detach partition	ALTER TABLE t DETACH PARTITION t_2022 CONCURRENTLY	Non-blocking in PG14+
Drop old partition	DROP TABLE t_2019	After detaching
Attach partition	ALTER TABLE t ATTACH PARTITION t_2022 FOR VALUES FROM (...) TO (...)	Validates existing rows
List partitions	SELECT * FROM pg_inherits JOIN pg_class ...	Or \d+ parent in psql
Check pruning	EXPLAIN SELECT ... WHERE partition_key = ...	Look for 'Partitions pruned'
Spring @Scheduled partition	@Scheduled(cron=...) void createNextPartition()	Use JdbcTemplate.execute()